

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
Σχολή Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών
Τεχνολογίες Υπηρεσιών Λογισμικού

Τεχνολογίες Υπηρεσιών Λογισμικού

Β.Βεσκούκης, Καθ.ΕΜΠ (v.vescoukis@cs.ntua.gr)

Microservices architectures using AI tools

General approach

- Can LLMs correctly apply requested architectural patterns and create useful architectural diagrams?
- How does requirements representation affect the ability of LLMs to apply architectural patterns?
- How does Retrieval-Augmented Generation (RAG) affect the quality of LLM-generated diagrams?
- Can LLMs reliably calculate quantitative metrics regarding the diagrams they generated, so that we can evaluate the output?

No LLM benchmarks for software arch



No empirical evidence

On how requirement representations influence pattern application across LLMs.



No representational consensus

Diagram-as-Code, images, XML, or Architecture-as-Code — no agreed format.



No evaluation standard

Qualitative and quantitative dimensions for assessing architectures aren't fixed.



No shared dataset

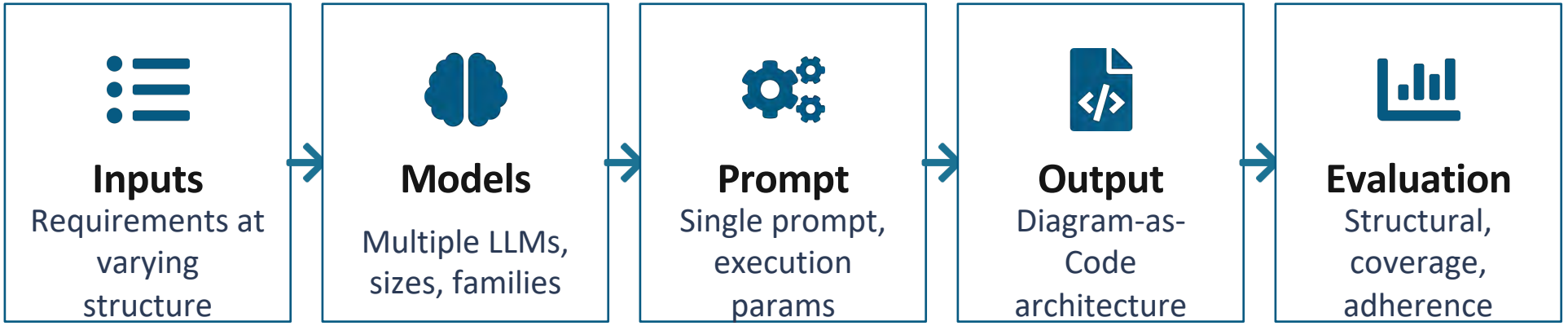
No common benchmark for reproducible LLM/prompt comparison across complexity levels.

Trial and error

Prompt an LLM with a problem definition

Examine the generated architecture

Refine as needed, or reject and try alternative problem definitions or LLM



Requirements: how to represent

Free-form text	Semi-structured list	Structured requirements	Full SRS document
Plain natural-language description of the system.	Bulleted requirement list, lightly normalized.	Categorized functional and non-functional sets.	Complete Software Requirements Specification (SRS).

LESS STRUCTURE → MORE STRUCTURE

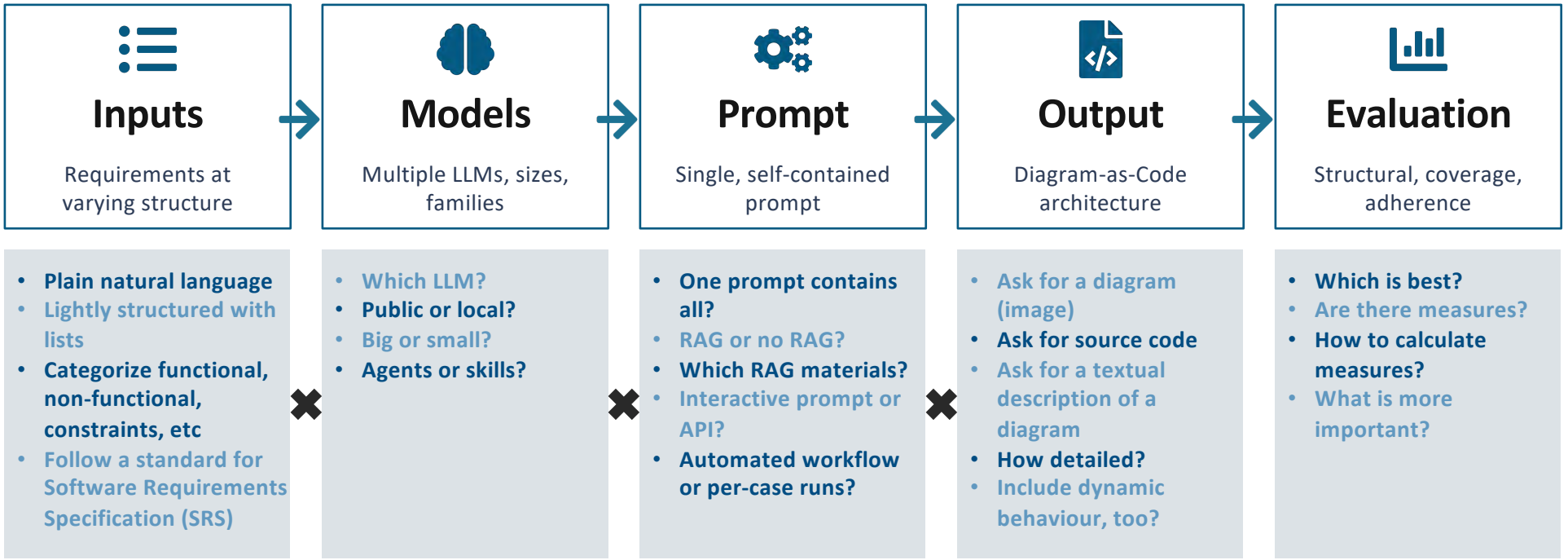
Assist models using RAG



Plus: Retrieval-Augmented Generation

Provide additional materials as RAG to guide LLMs to your context, terminology, guidelines, good practices, etc.

More choices to make



How to evaluate LLM outputs



Structural correctness

Are generated components, responsibilities, and interactions internally consistent?



Requirement coverage

Does the generated architecture actually address each provided requirement?



Pattern adherence

Does the result follow the constraints of the requested architectural pattern?



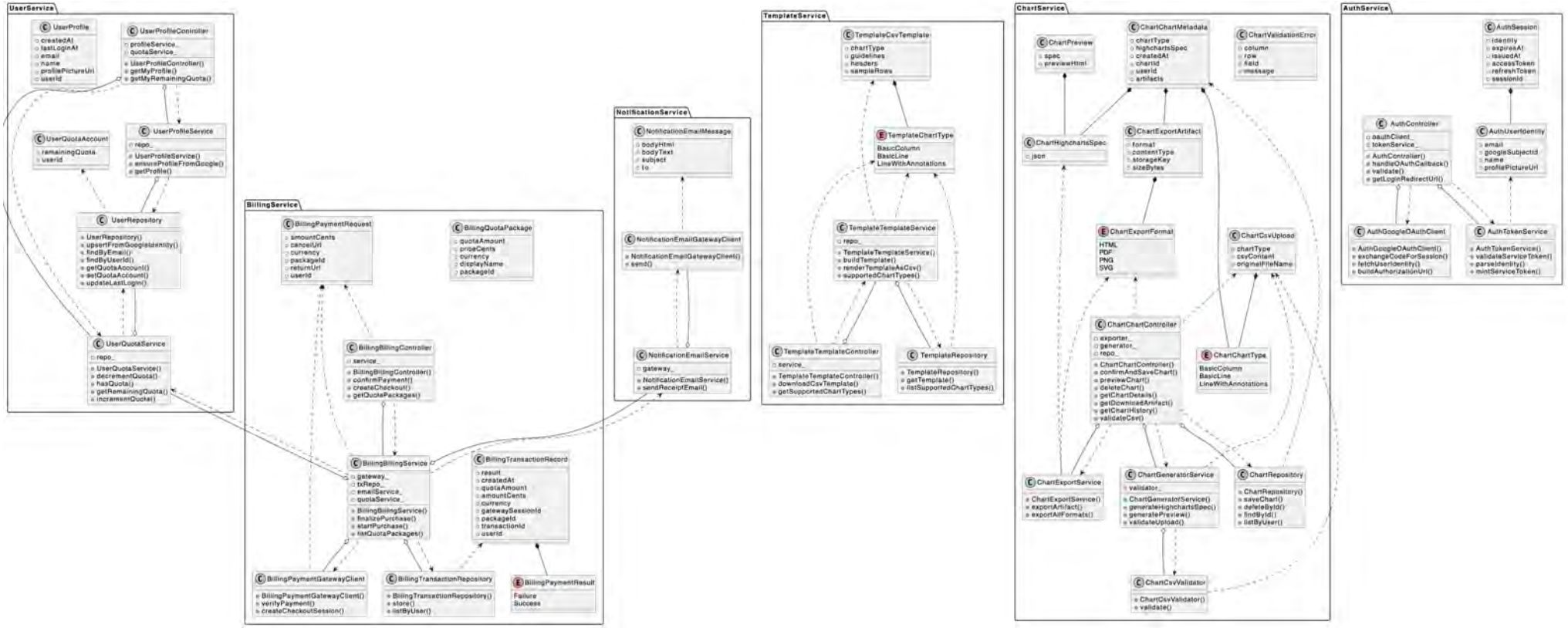
Self-reported metrics

Can the model produce reliable quantitative claims about its own generated diagram?

These dimensions separate "plausible-looking" from "genuinely well-structured."

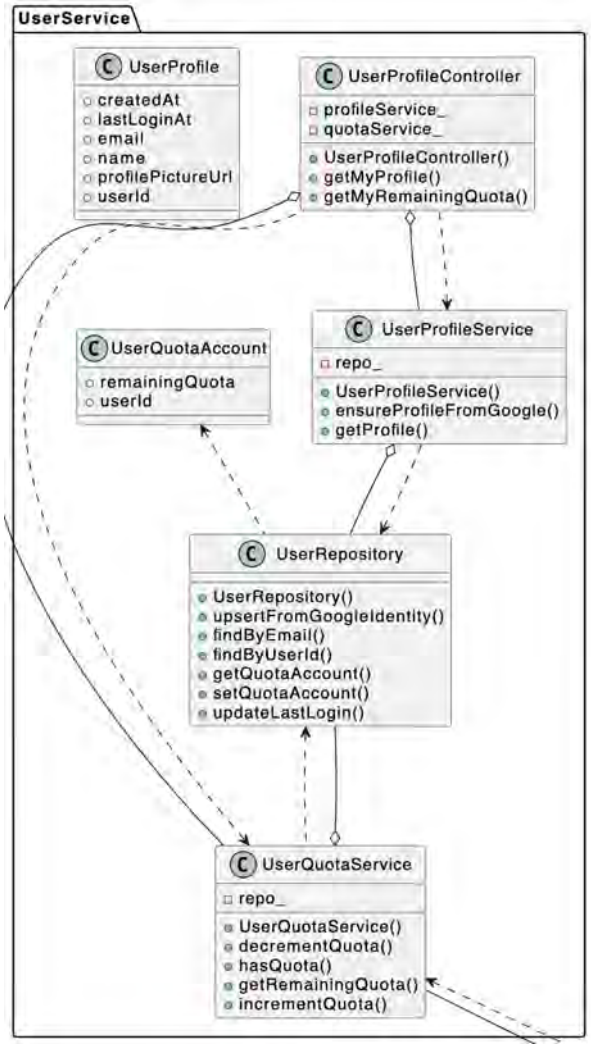
Representation of architectures from LLMs

PlantUML is good for diagrams



Possibly, any DaC will do, BUT PlantUML is more mature

PlantUML, basic




```
namespace UserService {
  class UserProfile {
    +createdAt
    +lastLoginAt
    +email
    +name
    +profilePictureUrl
    +userId
  }

  class UserProfileController {
    +UserProfileController()
    +getMyProfile()
    -profileService_
    -quotaService_
    +getMyRemainingQuota()
  }

  class UserProfileService {
    +UserProfileService()
    +ensureProfileFromGoogle()
    +getProfile()
    -repo_
  }

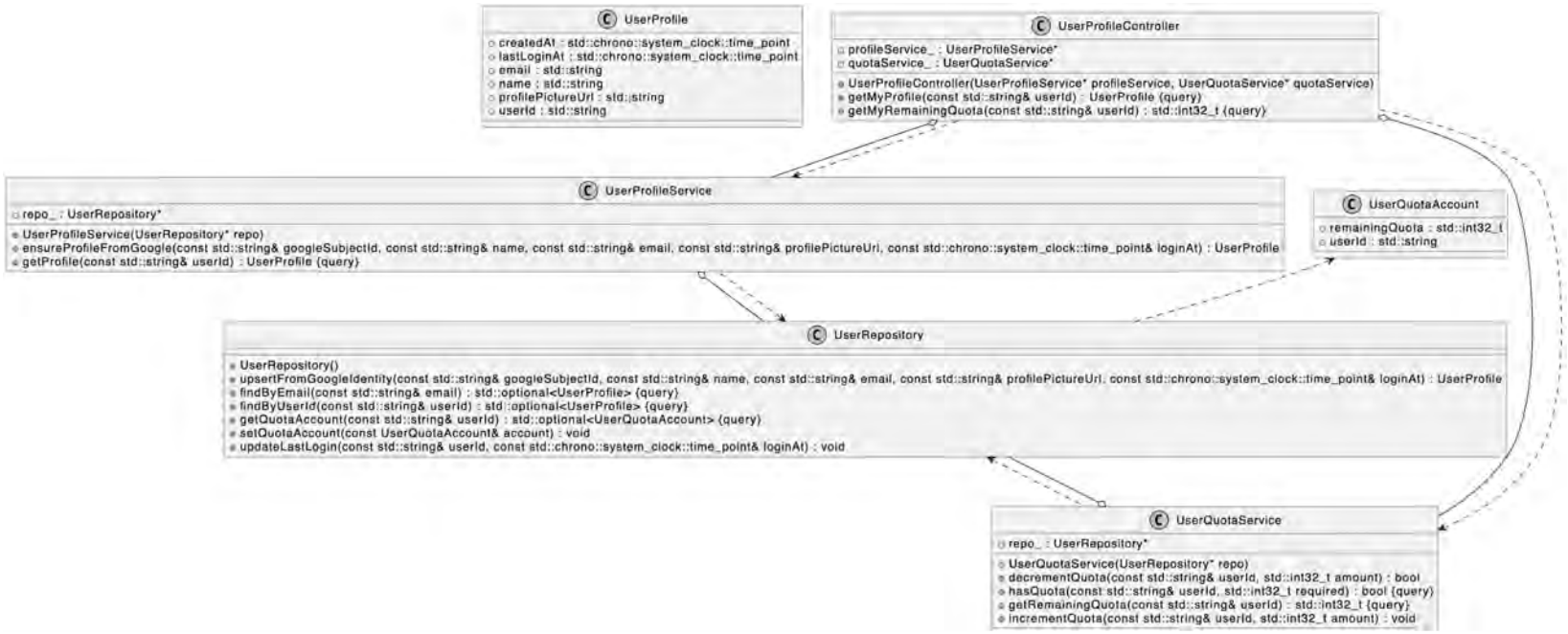
  class UserQuotaAccount {
    +remainingQuota
    +userId
  }

  class UserQuotaService {
    +UserQuotaService()
    -repo_
    +decrementQuota()
    +hasQuota()
    +getRemainingQuota()
    +incrementQuota()
  }

  class UserRepository {
    +UserRepository()
    +upsertFromGoogleIdentity()
    +findByEmail()
    +findByUserId()
    +getQuotaAccount()
    +setQuotaAccount()
    +updateLastLogin()
  }
}

UserService.UserProfileController o-- UserService.UserProfileService
UserService.UserProfileController o-- UserService.UserQuotaService
UserService.UserProfileService o-- UserService.UserRepository
UserService.UserQuotaService o-- UserService.UserRepository
UserService.UserProfileService <.. UserService.UserProfileController
UserService.UserQuotaAccount <.. UserService.UserRepository
UserService.UserQuotaService <.. UserService.UserProfileController
UserService.UserRepository <.. UserService.UserProfileService
UserService.UserRepository <.. UserService.UserQuotaService
```

PlantUML, complete



PlantUML, complete

```
@startuml
    class UserProfile {
        +createdAt : std::chrono::system_clock::time_point
        +lastLoginAt : std::chrono::system_clock::time_point
        +email : std::string
        +name : std::string
        +profilePictureUrl : std::string
        +userId : std::string
    }

    class UserProfileController {
        +UserProfileController(UserProfileService* profileService, UserQuotaService* quotaService)
        +getMyProfile(const std::string& userId) : UserProfile {query}
        -profileService_ : UserProfileService*
        -quotaService_ : UserQuotaService*
        +getMyRemainingQuota(const std::string& userId) : std::int32_t {query}
    }

    class UserProfileService {
        +UserProfileService(UserRepository* repo)
        +ensureProfileFromGoogle(const std::string& googleSubjectId, const std::string& name, const std::string& email, const
        std::string& profilePictureUrl, const std::chrono::system_clock::time_point& loginAt) : UserProfile
        +getProfile(const std::string& userId) : UserProfile {query}
        -repo_ : UserRepository*
    }

    class UserQuotaAccount {
        +remainingQuota : std::int32_t
        +userId : std::string
    }

    class UserQuotaService {
        +UserQuotaService(UserRepository* repo)
        -repo_ : UserRepository*
        +decrementQuota(const std::string& userId, std::int32_t amount) : bool
        +hasQuota(const std::string& userId, std::int32_t required) : bool {query}
        +getRemainingQuota(const std::string& userId) : std::int32_t {query}
        +incrementQuota(const std::string& userId, std::int32_t amount) : void
    }

    class UserRepository {
        +UserRepository()
        +upsertFromGoogleIdentity(const std::string& googleSubjectId, const std::string& name, const std::string& email, const
        std::string& profilePictureUrl, const std::chrono::system_clock::time_point& loginAt) : UserProfile
        +findByEmail(const std::string& email) : std::optional<UserProfile> {query}
        +findByUserId(const std::string& userId) : std::optional<UserProfile> {query}
        +getQuotaAccount(const std::string& userId) : std::optional<UserQuotaAccount> {query}
        +setQuotaAccount(const UserQuotaAccount& account) : void
        +updateLastLogin(const std::string& userId, const std::chrono::system_clock::time_point& loginAt) : void
    }

    /' Aggregation relationships '/
    UserProfileController o-- UserProfileService
    UserProfileController o-- UserQuotaService
    UserProfileService o-- UserRepository
    UserQuotaService o-- UserRepository

    /' Dependency relationships '/
    UserProfileService <.. UserProfileController
    UserQuotaAccount <.. UserRepository
    UserQuotaService <.. UserProfileController
    UserRepository <.. UserProfileService
    UserRepository <.. UserQuotaService
    @enduml
```

Question:

How to make LLMs generate and use types?

PlantUML is ok for presentation, but…

1. Ask LLM to generate C++ headers

```cpp
class UserProfile final {
public:
  std::string userId; // internal user id
  std::string name;
  std::string email;
  std::string profilePictureUrl;
  std::chrono::system_clock::time_point createdAt{};
  std::chrono::system_clock::time_point lastLoginAt{};
};

class UserProfileController final {
public:
  UserProfileController(UserProfileService* profileService, UserQuotaService* quotaService);

  UserProfile getMyProfile(const std::string& userId) const;
  std::int32_t getMyRemainingQuota(const std::string& userId) const;

private:
  UserProfileService* profileService_{};
  UserQuotaService* quotaService_{};
};

class UserProfileService final {
public:
  explicit UserProfileService(UserRepository* repo);

  UserProfile getProfile(const std::string& userId) const;
  UserProfile ensureProfileFromGoogle(const std::string& googleSubjectId,
                                      const std::string& name,
                                      const std::string& email,
                                      const std::string& profilePictureUrl,
                                      const std::chrono::system_clock::time_point& loginAt);

private:
  UserRepository* repo_{};
};

class UserQuotaAccount final {
public:
  std::string userId;
  std::int32_t remainingQuota{0};
};
```

2. Convert (transpile) C++ to PlantUML

```
@startuml
    class UserProfile {
        +createdAt : std::chrono::system_clock::time_point
        +lastLoginAt : std::chrono::system_clock::time_point
        +email : std::string
        +name : std::string
        +profilePictureUrl : std::string
        +userId : std::string
    }

    class UserProfileController {
        +UserProfileController(UserProfileService* profileService, UserQuotaService* quotaService)
        +getMyProfile(const std::string& userId) : UserProfile {query}
        -profileService_ : UserProfileService*
        -quotaService_ : UserQuotaService*
        +getMyRemainingQuota(const std::string& userId) : std::int32_t {query}
    }

    class UserProfileService {
        +UserProfileService(UserRepository* repo)
        +ensureProfileFromGoogle(const std::string& googleSubjectId, const std::string& name, const std::string& email, const
        std::string& profilePictureUrl, const std::chrono::system_clock::time_point& loginAt) : UserProfile
        +getProfile(const std::string& userId) : UserProfile {query}
        -repo_ : UserRepository*
    }

    class UserQuotaAccount {
        +remainingQuota : std::int32_t
        +userId : std::string
    }
```

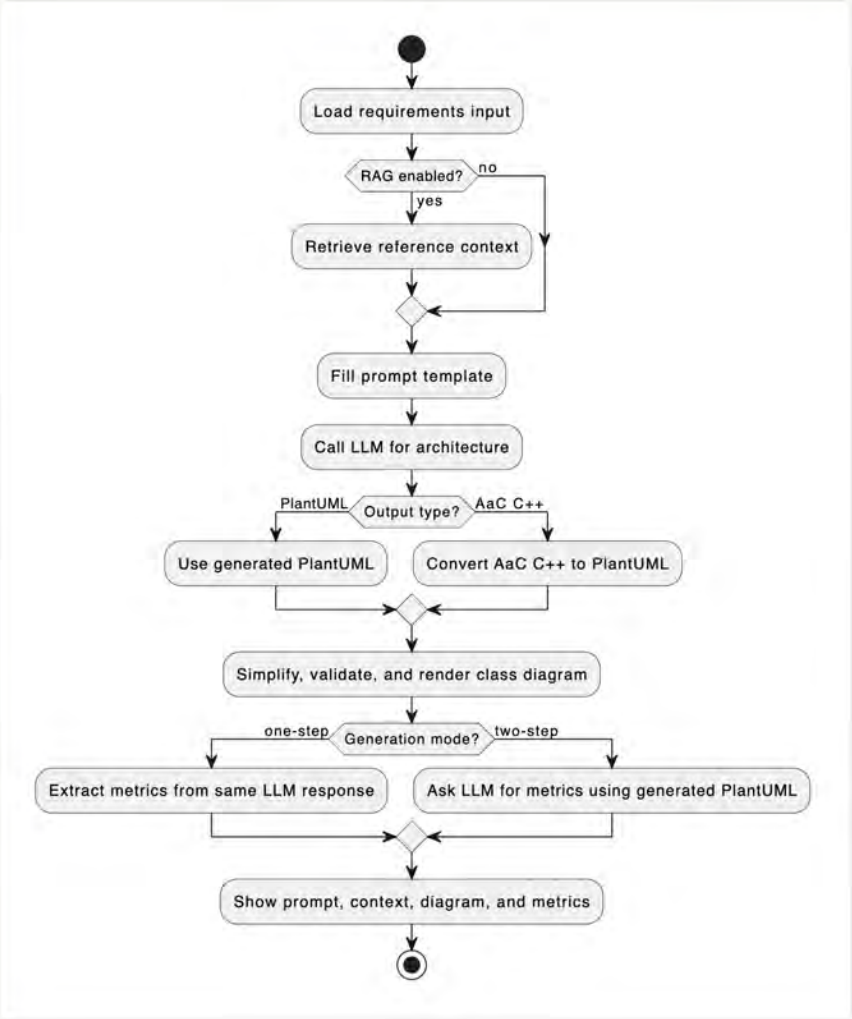
Examples

More available at https://davinci.softlab.ntua.gr/saasllmarch/

Microservices with LLMs

Pipeline Flow

1. **Load inputs**
   Use either FR/NFR files or one SRS file.
2. **Add context**
   Optional LangChain RAG over reference documents.
3. **Fill prompt**
   Patch requirements, architecture, output type, and metrics instructions into templates.
4. **Generate architecture**
   Ask the selected LLM for PlantUML or C++ Architecture-as-Code.
5. **Render diagram**
   Convert AaC if needed, then simplify, validate, and render the class diagram.
6. **Calculate metrics**
   One-step extracts JSON from the same response; two-step sends a second metrics prompt.



Microservices with LLMs examples

Experiment Overview

RUNS	INPUTS	MODELS
48 generated examples	FR/NFR or SRS	OpenAI + DeepSeek configs

OUTPUT	EXECUTION	CONTEXT
PlantUML or AaC C++	One-step or two-step	RAG or no RAG

DIMENSION	CHOICES	PURPOSE
Input form	FR/NFR, SRS	Compare structured requirements with full specification text.
Architecture output	PlantUML, AaC C++	Compare direct diagram generation with architecture-as-code conversion.
Metrics strategy	One-step, two-step	Compare metrics from the same response with a second focused LLM call.
Grounding	RAG, no RAG	Show how retrieved reference material changes the generated design.

Microservices with LLMs examples

```
Functional requirements:
- Authenticate users via Google accounts.
- Allow download of CSV templates for 3 supported chart types (Basic line, Line with annotations, Basic column).
- Upload CSV file with the user data for chart generation.
- Validate uploaded CSV files against the template structure, data types, and mandatory data existence.
- Generate charts with data from the CSV file uploaded by the user, using Highcharts.
- Save charts to the server in PDF, PNG, SVG, and HTML formats.
- Display a dashboard with the history of user-generated charts, including chart previews.
- Download selected chart type.
- Charge quotas for chart creation.
- Allow users to delete or download charts.
- Sell quotas and receive payment by a payment gateway
- Maintain user profiles containing the following data: Name (from Google account), profile picture (from Google
account), email (from Google account), remaining quota, account creation timestamp, last login timestamp.
- Display user's profile info, including remaining quotas.


Non-functional requirements:
### Performance

- Chart generation completes within 3 seconds.
  Usability
- Intuitive UI with guided workflows (e.g., tooltips, validation hints) to assist non-technical users.

### Security
- SSL encryption: Data transmitted ove the internet is encrypted using SSL.

### Availability
- The service should be available 90% of the time on a daily basis.
- The system should prioritize Availability over Consistency during network partitioning.
- While in network partitioning, although temporary data inconsistencies may occur, as many as possible services should
remain fully operational. Scalability
  Support 1,000 concurrent users.
```

Microservices with LLMs examples

Prompt Template

```
Task Description: You are tasked with processing a software description and its requirements to generate a class diagram using PlantUML. The class diagram
should respect the requested software architecture, define all necessary classes, and accurately represent associations between them.

The requirements for the software we want you to design are described below:

Functional requirements:
{FR}

Non-functional requirements:
{NFR}

The demanded Architecture is: {ARCHITECTURE}

Additional retrieval context:
{RAG_CONTEXT}

Important guidelines for designing proper microservices architecture:

1. Functional Alignment & Responsibility Distribution
Focus: Ensure that each microservice maps to a bounded context and implements a focused set of functionalities. The union of the functionalities of each
microservice should be the entire set of functional requirements.

2. Coupling & Deployment Independence
Focus: Ensure that microservices are loosely coupled and independently deployable.

3. Cohesion
Focus: Ensure high cohesion, meaning that more than one microservice needs to be involved in the completion of each use case.

4. Data Management
Focus: Ensure that each service is responsible for managing specific data elements without using shared databases.

5. Data Consistency
Focus: Ensure that operations to achieve eventual consistency of data states among different microservices are included.

6. Communication & Flow Control
Focus: Ensure that service coordination is done using choreography or orchestration mechanisms. API gateways and/or messaging services should be used to
implement flow control.

7. Non-Functional Requirements
Focus: Ensure that the design satisfies all problem-specific non-functional requirements.
```

Microservices with LLMs examples

```
First, generate a class diagram using PlantUML that defines all necessary classes and associations based on the described architecture, requirements, and
functionality. Ensure that the diagram reflects separation of concerns, defines attributes and methods, includes associations and service dependencies, and
creates appropriate packages.

PlantUML class diagram requirements:
- The output must start with @startuml and end with @enduml.
- Use only class-diagram constructs: package, class, interface, enum, abstract class, associations, dependencies, aggregations, compositions, and
inheritance.
- Every package must have a quoted or valid name, for example: package "Chart Service" { ... }.
- Do not output anonymous packages such as "package {" or a line containing only "package".
- Include explicit relationships between classes/services using PlantUML relationship arrows such as -->, ..>, o--, *--, <|--.
- Include at least one relationship for every service class unless it is truly isolated.
- Do not generate sequence diagram syntax such as participant, actor, -> messages, activate, deactivate, or lifeline calls.

Classify each operation by commenting next to the operation in PlantUML with one of these groups:
- Functional requirement-business logic operation
- Data management operation
- Data consistency operation
- Flow control operation

Then calculate the architecture metrics.

Respond exactly in this order:

ARCHITECTURE_OUTPUT_START
<PlantUML here, including @startuml and @enduml>
ARCHITECTURE_OUTPUT_END

METRICS_JSON_START
<metrics JSON here>
METRICS_JSON_END

Metrics instructions:

{METRICS_PROMPT}
```

Microservices with LLMs examples

Filled Prompt

```
Task Description: You are tasked with processing a software description and its requirements to generate a class diagram using PlantUML. The class diagram
should respect the requested software architecture, define all necessary classes, and accurately represent associations between them.

The requirements for the software we want you to design are described below:

Functional requirements:
- Authenticate users via Google accounts.
- Allow download of CSV templates for 3 supported chart types (Basic line, Line with annotations, Basic column).
- Upload CSV file with the user data for chart generation.
- Validate uploaded CSV files against the template structure, data types, and mandatory data existence.
- Generate charts with data from the CSV file uploaded by the user, using Highcharts.
- Save charts to the server in PDF, PNG, SVG, and HTML formats.
- Display a dashboard with the history of user-generated charts, including chart previews.
- Download selected chart type.
- Charge quotas for chart creation.
- Allow users to delete or download charts.
- Sell quotas and receive payment by a payment gateway
- Maintain user profiles containing the following data: Name (from Google account), profile picture (from Google account), email (from Google account),
remaining quota, account creation timestamp, last login timestamp.
- Display user's profile info, including remaining quotas.


Non-functional requirements:
### Performance

- Chart generation completes within 3 seconds.
  Usability
- Intuitive UI with guided workflows (e.g., tooltips, validation hints) to assist non-technical users.

### Security

- SSL encryption: Data transmitted ove the internet is encrypted using SSL.

### Availability

- The service should be available 90% of the time on a daily basis.
- The system should prioritize Availability over Consistency during network partitioning.
- While in network partitioning, although temporary data inconsistencies may occur, as many as possible services should remain fully operational.
Scalability
  Support 1,000 concurrent users.
```

Microservices with LLMs examples

Filled Prompt

```
The demanded Architecture is: microservices

Additional retrieval context:
[Source: rag/reference_docs/rag1.pdf]
tectural scenarios.
DEFINING SYSTEM OPERATIONS
Once you've defined a high-level domain model, the next step is to identify the requests
that the application must handle. The details  of the UI are beyond the scope of this
book, but you can imagine that in each user scenario, the UI will make requests to the
backend business logic to retrieve and upda te data. FTGO is primarily a web applica-
tion, which means that most requests are HTTP-based, but it's possible that some clients
might use messaging. Instead of committing to a specific protocol, therefore, it makes
sense to use the more abstract notion of a system operation to represent requests.
 There are two types of system operations:
 Commands—System operations that create, update, and delete data
 Queries—System operations that read (query) data
Ultimately, these system operations will correspond to REST, RPC, or messaging
endpoints, but for now thinking of them abst ractly is useful. Let's first identify some
commands.
```

```
Important guidelines for designing proper microservices architecture:

1. Functional Alignment & Responsibility Distribution
Focus: Ensure that each microservice maps to a bounded context and implements a focused set of functionalities. The union of the functionalities of each
microservice should be the entire set of functional requirements.

2. Coupling & Deployment Independence
Focus: Ensure that microservices are loosely coupled and independently deployable.

3. Cohesion
Focus: Ensure high cohesion, meaning that more than one microservice needs to be involved in the completion of each use case.

4. Data Management
Focus: Ensure that each service is responsible for managing specific data elements without using shared databases.

5. Data Consistency
Focus: Ensure that operations to achieve eventual consistency of data states among different microservices are included.

6. Communication & Flow Control
Focus: Ensure that service coordination is done using choreography or orchestration mechanisms. API gateways and/or messaging services should be used to
implement flow control.

7. Non-Functional Requirements
Focus: Ensure that the design satisfies all problem-specific non-functional requirements.

First, generate a class diagram using PlantUML that defines all necessary classes and associations based on the described architecture, requirements, and
functionality. Ensure that the diagram reflects separation of concerns, defines attributes and methods, includes associations and service dependencies, and
creates appropriate packages.
```

Microservices with LLMs examples

```
PlantUML class diagram requirements:
- The output must start with @startuml and end with @enduml.
- Use only class-diagram constructs: package, class, interface, enum, abstract class, associations, dependencies, aggregations, compositions, and
inheritance.
- Every package must have a quoted or valid name, for example: package "Chart Service" { ... }.
- Do not output anonymous packages such as "package {" or a line containing only "package".
- Include explicit relationships between classes/services using PlantUML relationship arrows such as -->, ..>, o--, *--, <|--.
- Include at least one relationship for every service class unless it is truly isolated.
- Do not generate sequence diagram syntax such as participant, actor, -> messages, activate, deactivate, or lifeline calls.

Classify each operation by commenting next to the operation in PlantUML with one of these groups:
- Functional requirement-business logic operation
- Data management operation
- Data consistency operation
- Flow control operation

Then calculate the architecture metrics.

Respond exactly in this order:

ARCHITECTURE_OUTPUT_START
<PlantUML here, including @startuml and @enduml>
ARCHITECTURE_OUTPUT_END

METRICS_JSON_START
<metrics JSON here>
METRICS_JSON_END

Metrics instructions:

You are tasked with calculating architecture metrics from a generated PlantUML architecture.

For two-step runs, use the PlantUML provided below.
For one-step runs, use the architecture generated earlier in the same response.
```

Microservices with LLMs examples

```
Architecture style:
microservices

Generated PlantUML:
Use the architecture generated above in this same response.

Calculate the following characteristics of the architecture.

General characteristics:

SI: Counts services that do not carry persistent state across requests and divides by the total number of services.
SI = (# of stateless services) / |Total number of services|

DOC (Data Ownership Count): Counts services that manage a dedicated data store and divides by the total number of services.
DOC = (# of services linked to a dedicated data store) / |Total number of services|

SST (Service Support for Transactions): Counts services that have at least one operation classified as "Data consistency operation" and divides by the total
number of services.
SST = (# of transaction-aware services) / |Total number of services|

Characteristics per service:

SIC (Service Interface Count): Number of interfaces of service S.
AIS(in): Number of distinct consumers of service S.
ADS(out): Number of distinct services called by service S.

Characteristics per use case:

SC: Set of services involved for the completion of one use case.
SC = {{(Set of microservices involved per use case) MS SET}}

Use only the requirements and PlantUML above. If a value must be inferred, infer it conservatively from explicit services, operations, associations, and
operation comments.
```

Microservices with LLMs examples

LLM Generation Response

```
ARCHITECTURE_OUTPUT_START
@startuml
skinparam packageStyle rectangle
skinparam classAttributeIconSize 0

package "API Gateway" {
  class ApiGateway {
    +routeRequest() : void  ' Flow control operation
    +enforceSsl() : void  ' Flow control operation
  }

  interface ChartWebApi {
    +downloadCsvTemplate(chartType: ChartType) : CsvTemplate  ' Functional requirement-business logic operation
    +uploadCsv(file: UploadedFile) : UploadReceipt  ' Functional requirement-business logic operation
    +generateChart(request: ChartGenerationRequest) : ChartJob  ' Functional requirement-business logic operation
    +listDashboard(userId: UUID) : DashboardView  ' Functional requirement-business logic operation
    +downloadChart(chartId: UUID, format: ExportFormat) : BinaryFile  ' Functional requirement-business logic operation
    +deleteChart(chartId: UUID) : void  ' Data management operation
    +buyQuota(packageId: String) : PaymentSession  ' Functional requirement-business logic operation
    +getProfile(userId: UUID) : UserProfileView  ' Functional requirement-business logic operation
  }
}

package "Authentication Service" {
  class AuthService {
    +authenticateWithGoogle(idToken: String) : AuthSession  ' Functional requirement-business logic operation
    +refreshSession(refreshToken: String) : AuthSession  ' Data management operation
    +validateSession(sessionToken: String) : boolean  ' Flow control operation
  }

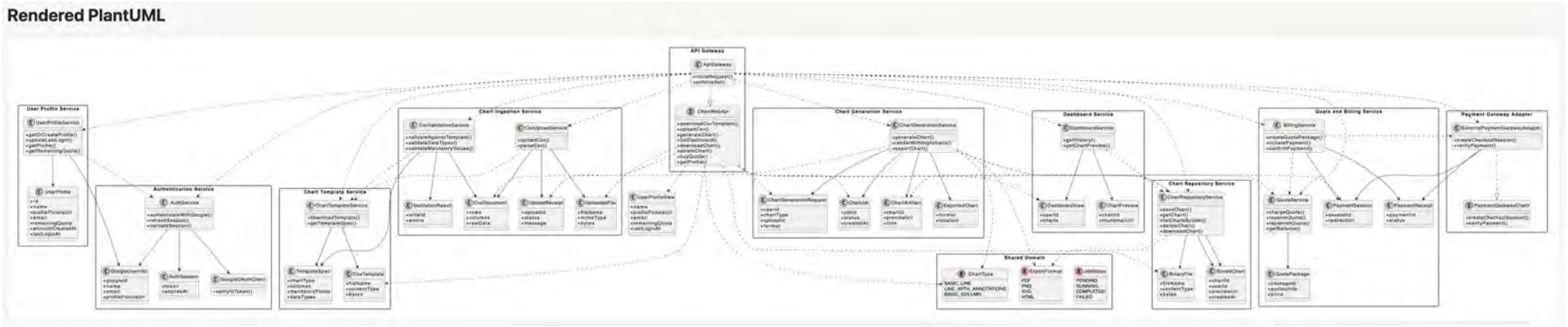
  class GoogleOAuthClient {
    +verifyIdToken(idToken: String) : GoogleUserInfo  ' Functional requirement-business logic operation
  }
```

Microservices with LLMs examples

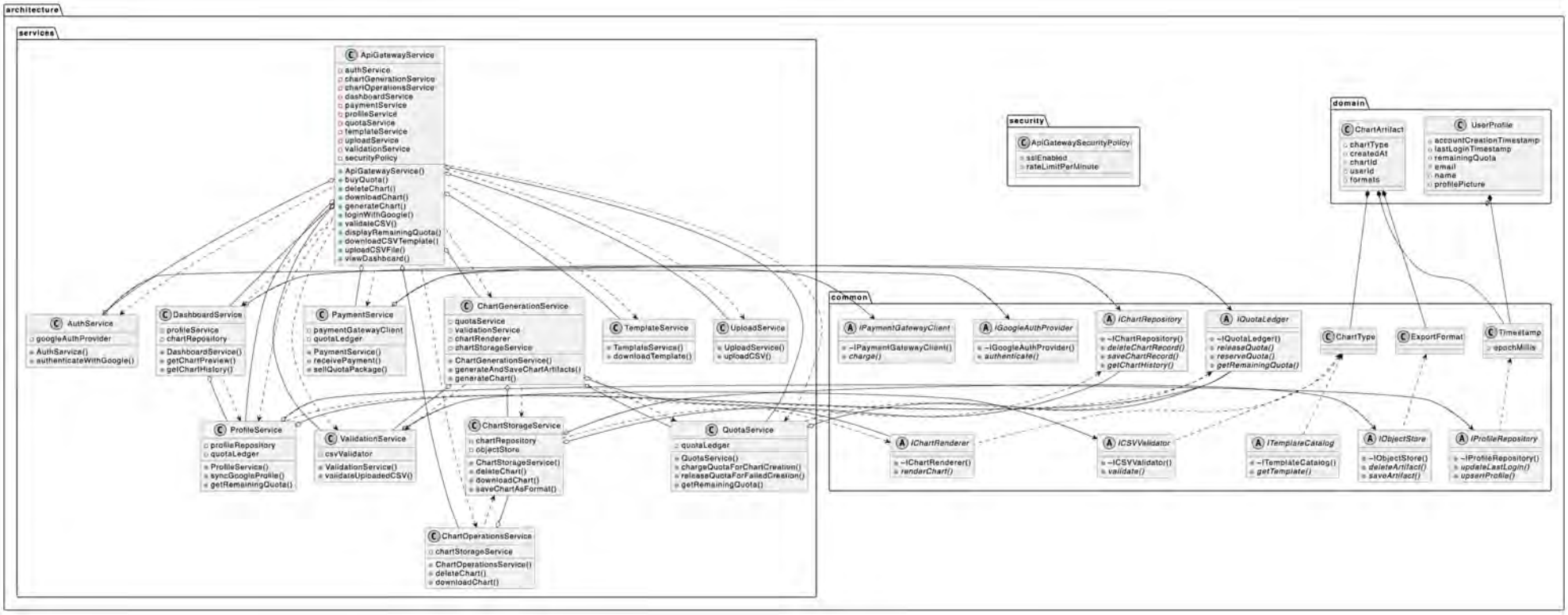
Metrics Response


```
{
  "SI": "0.5",
  "DOC": "0.5",
  "SST": "0.1666666667",
  "services": [
    {
      "service_name": "ApiGateway",
      "SIC": "1",
      "AIS(in)": "0",
      "ADS(out)": "10"
    },
    {
      "service_name": "AuthService",
      "SIC": "0",
      "AIS(in)": "2",
      "ADS(out)": "1"
    },
    {
      "service_name": "UserProfileService",
      "SIC": "0",
      "AIS(in)": "2",
      "ADS(out)": "1"
    },
    {
      "service_name": "ChartTemplateService",
      "SIC": "0",
      "AIS(in)": "2",
      "ADS(out)": "0"
    },
    {
      "service_name": "CsvUploadService",
      "SIC": "0",
      "AIS(in)": "2",
      "ADS(out)": "0"
    },
```

Microservices with LLMs examples



Microservices with LLMs examples

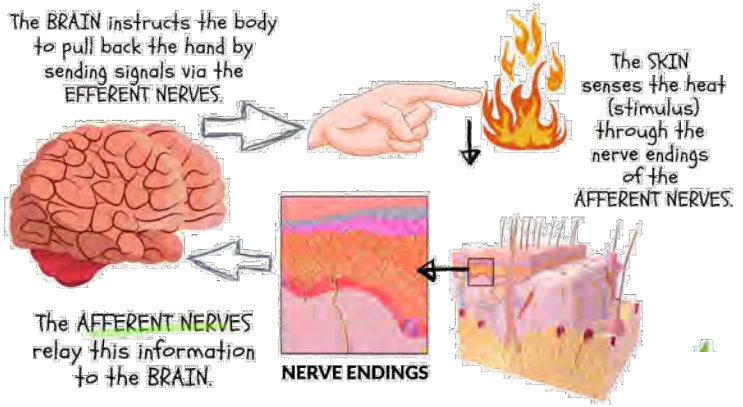


Architecture-related metrics

Metrics for coupling

Afferent Coupling (Ca)

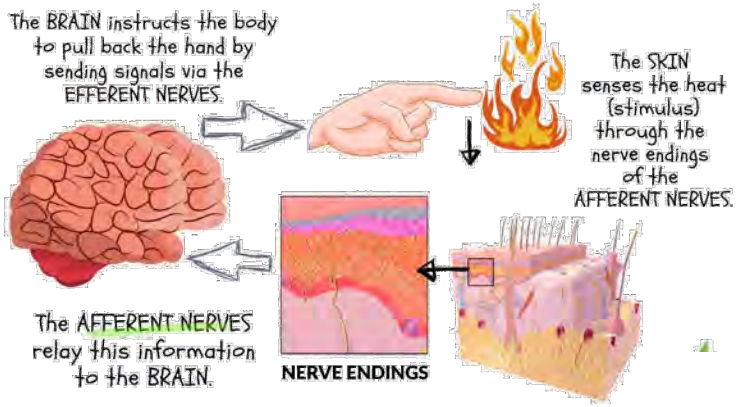
- The number of distinct packages that depend on a given package (use a service or a type in the package as parameter, return value or data type)
- A **high Ca** value indicates that the package is heavily relied upon by other parts of the system, making it a critical component



Metrics for coupling

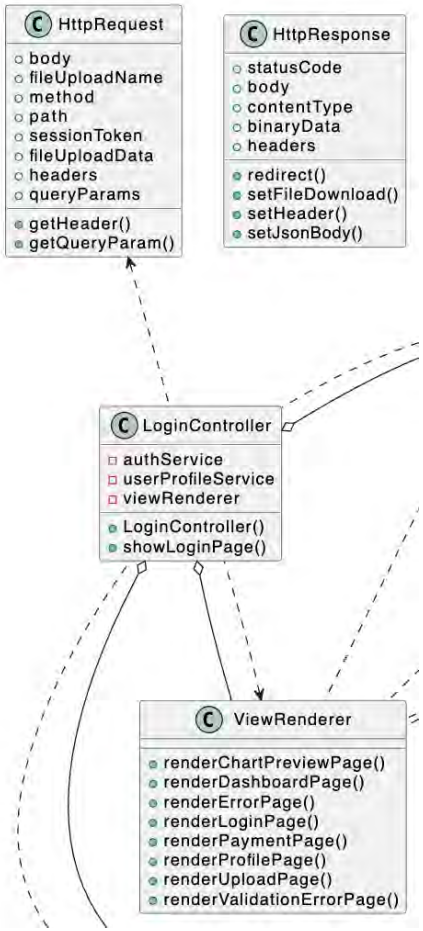
Efferent Coupling (Ce)

- The number of distinct packages a given package depends on (by using a service or a type as parameter, return value or data type)
- A **high Ce** value indicates that the package s sensitive to changes in other packages, making it fragile and potentially difficult to reuse in isolation

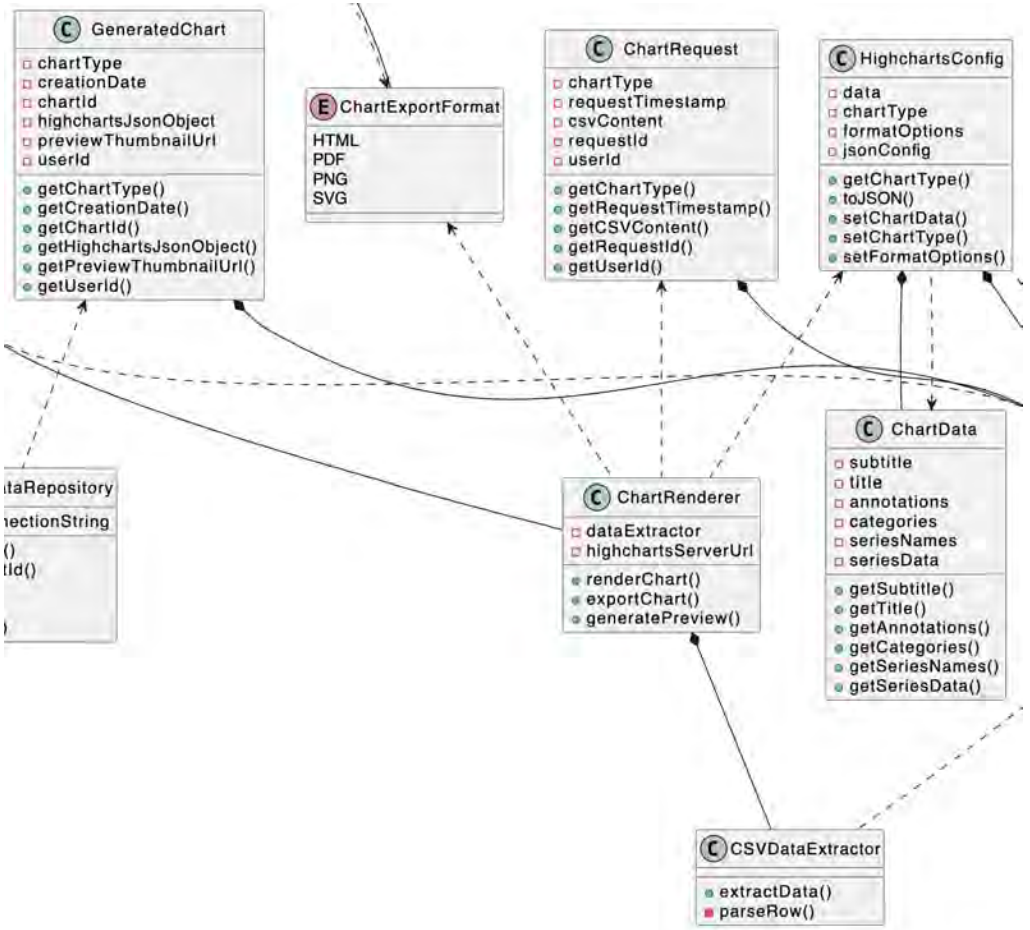


Metrics, examples

Example 1 (abstract)

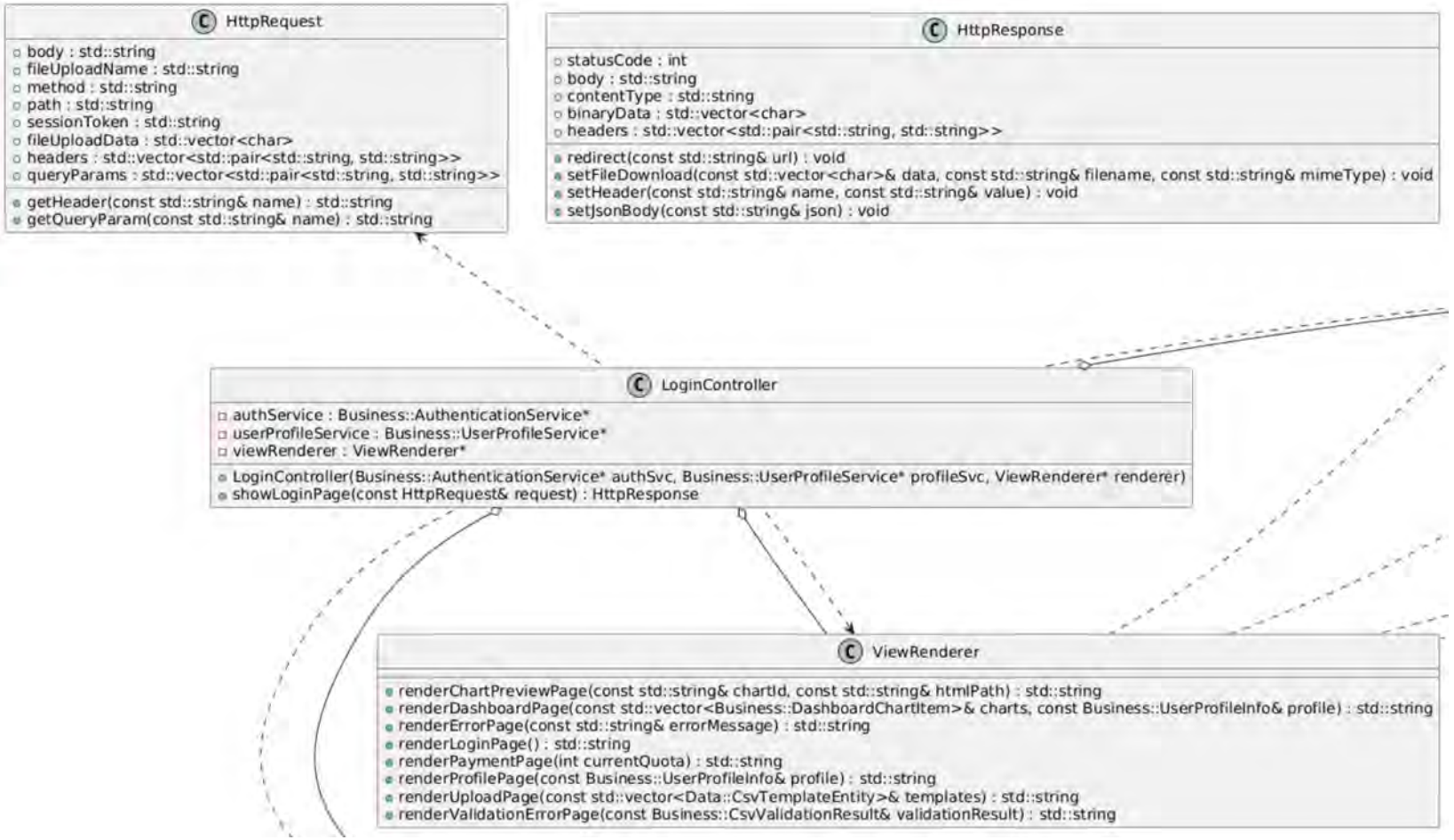


Example 2 (abstract)



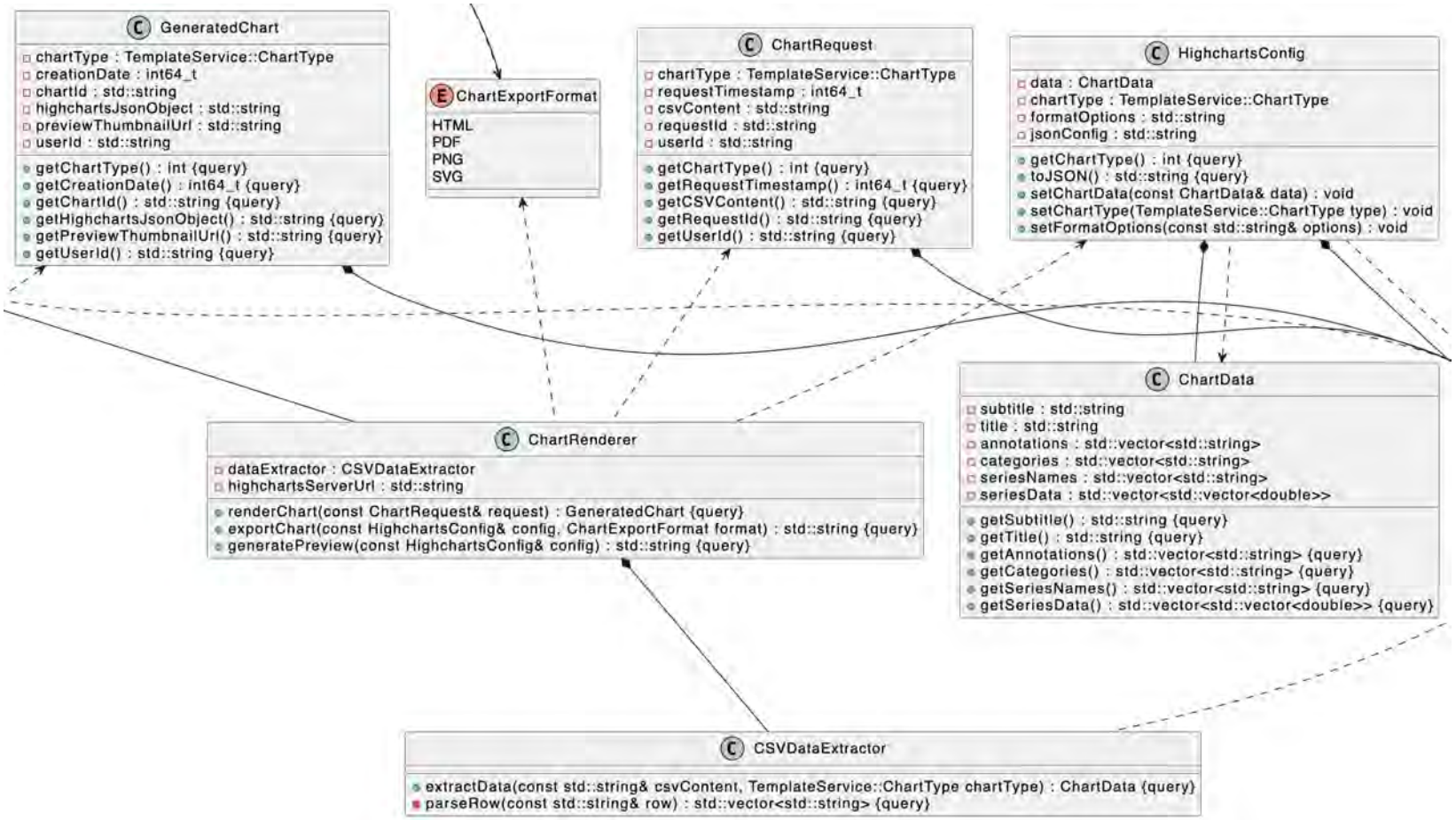
Metrics, examples

Example 1 (detail)



Metrics, examples

Example 2 (detail)



Metrics for coupling

Instability metric [0, 1]

$$I = \frac{C_e}{C_a + C_e}$$

- 0 indicates a maximally **stable** package: one that is depended upon by others but depends on nothing itself
- 1 indicates a maximally **unstable** package: one that depends on many others but is depended upon by none

Neither extreme is inherently desirable in isolation:
The goal is for each package to exhibit a level of instability appropriate to its architectural role

Metrics

Instability does not by itself indicate whether a package is well-designed; a package...

- can be maximally stable (I = 0) and still be poorly designed if it is entirely concrete (if it contains no abstract classes or interfaces). It is both heavily depended upon and impossible to extend without modification, violating the Open/Closed Principle.
- can be maximally unstable (I = 1) and be entirely abstract, which means it defines contracts that nothing implements and on which nothing depends, a situation equally undesirable.

Metrics

Abstractness metric

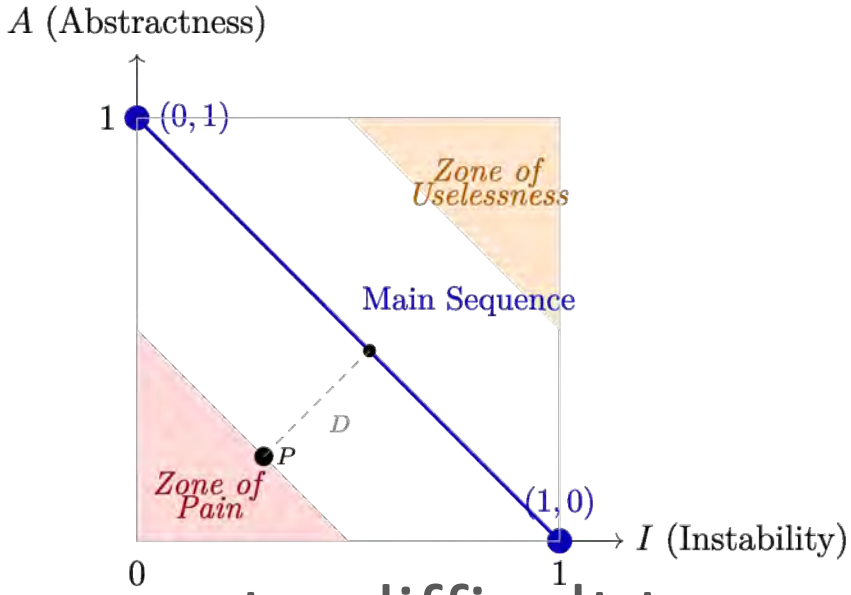
$$A = \frac{N_{abstract}}{N_{total}}$$

- Nabstract is the number of abstract classes or interfaces in the package
- Ntotal is the total number of classes

Metrics: the "Main Sequence" concept

Main Sequence:

an idealized line in the (I, A) plane defined by **A + I = 1**

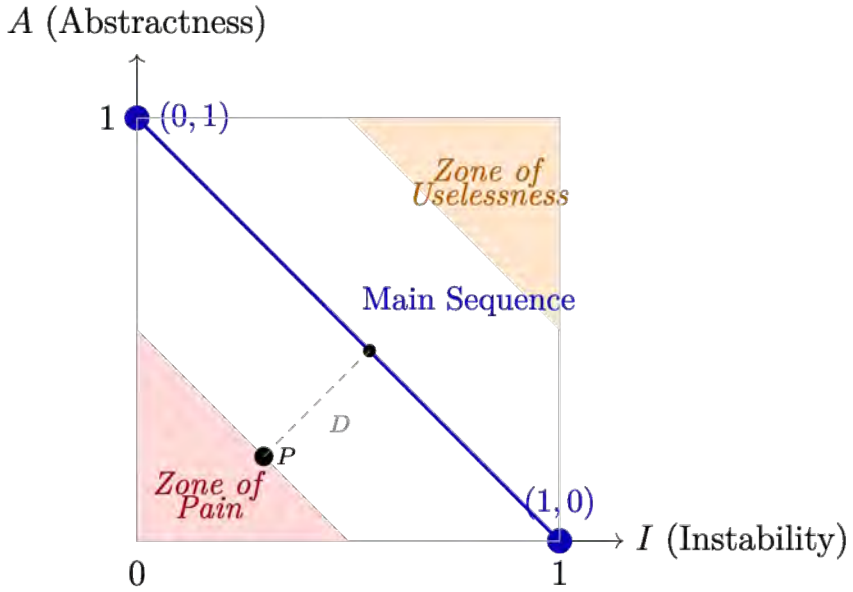


- Zone of Pain (low I, low A): packages that are stable and concrete, difficult to change and impossible to extend
- Zone of Uselessness (high I, high A): packages that are abstract but unstable, defining contracts that are not depended upon by anyone

Metrics: Distance

Distance

$$D = |A + I - 1|$$



- D = 0 indicates that the package lies exactly on the Main Sequence
- D > 0 indicate increasing deviation

- Heuristic measure, does not indicate a defect

Metrics: cohesion

Cohesion captures the internal organisation of a package, specifically, how strongly the classes within a package are related to one another

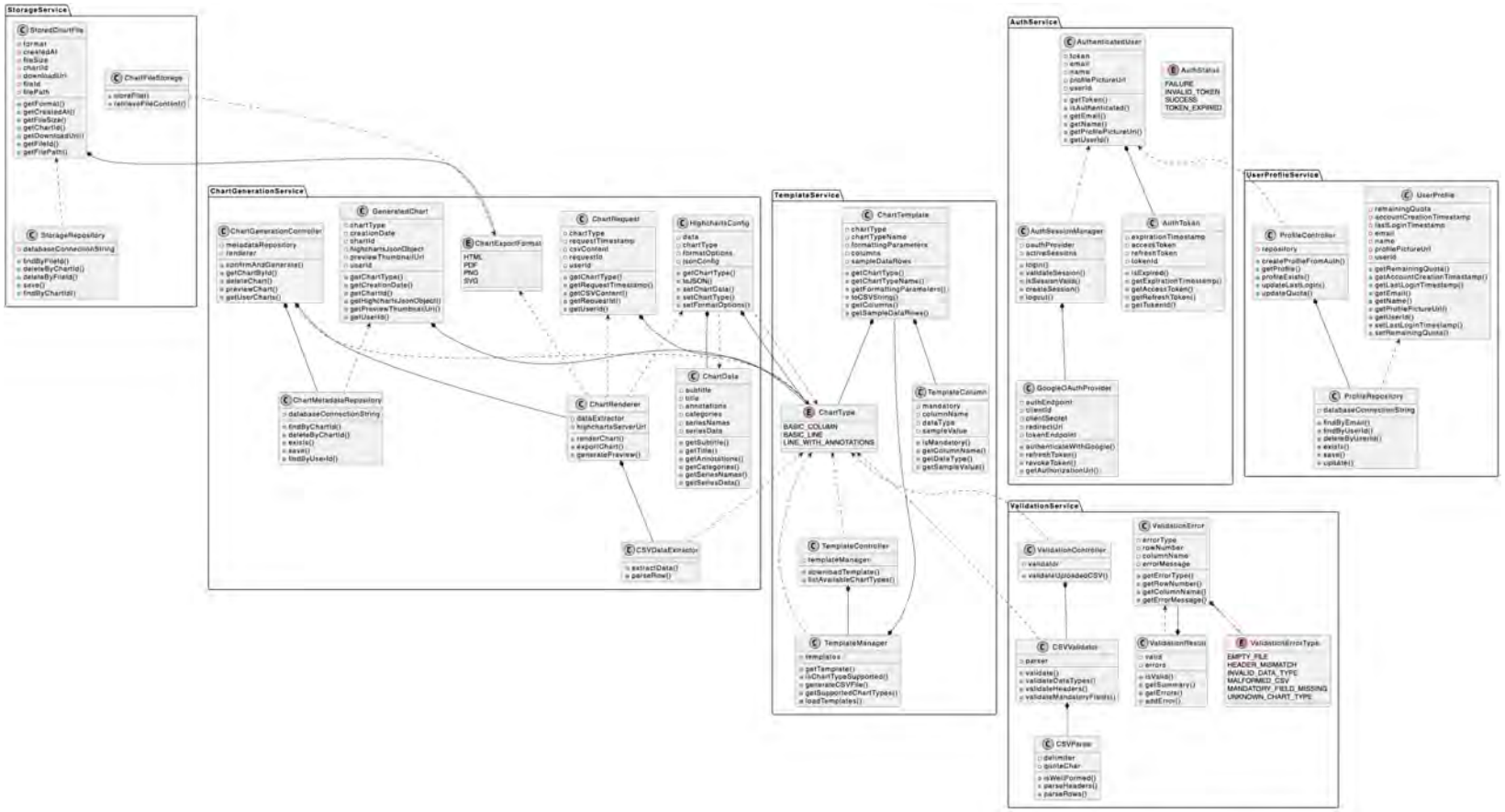
$$H = \frac{\zeta + 1}{N_c} \qquad H = \frac{\zeta}{\frac{N_c(N_c-1)}{2}}$$

Nc: number of classes in the package

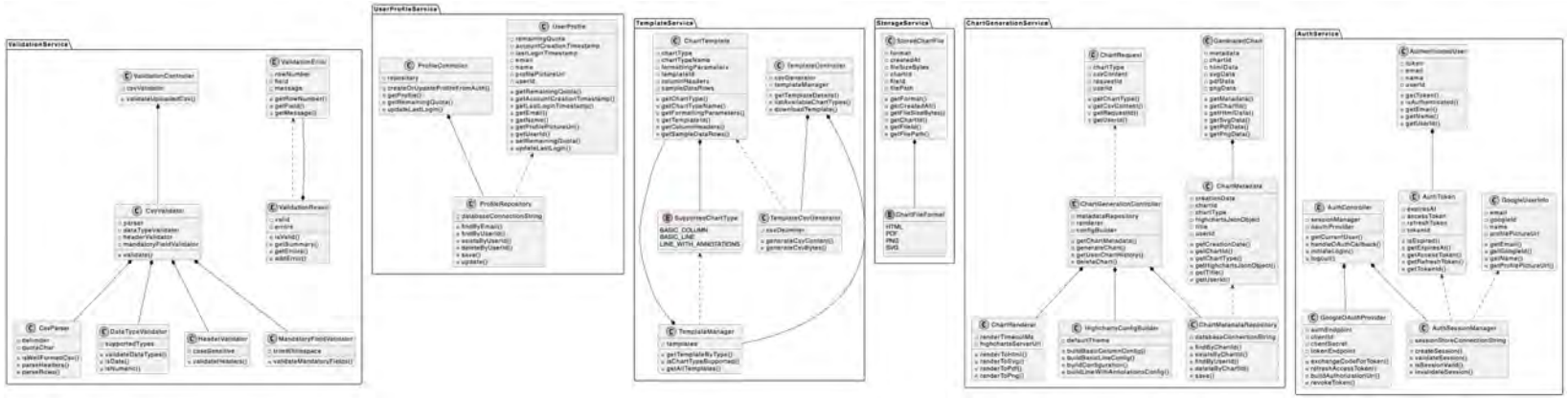
Z: number of dependency relationships among them

NOTE: a low cohesion does not necessarily indicate a design defect; it may reflect a deliberate architectural decision, such as a utility namespace containing loosely related helper classes. Cohesion should be interpreted in conjunction with the structural role of each package within the target architecture.

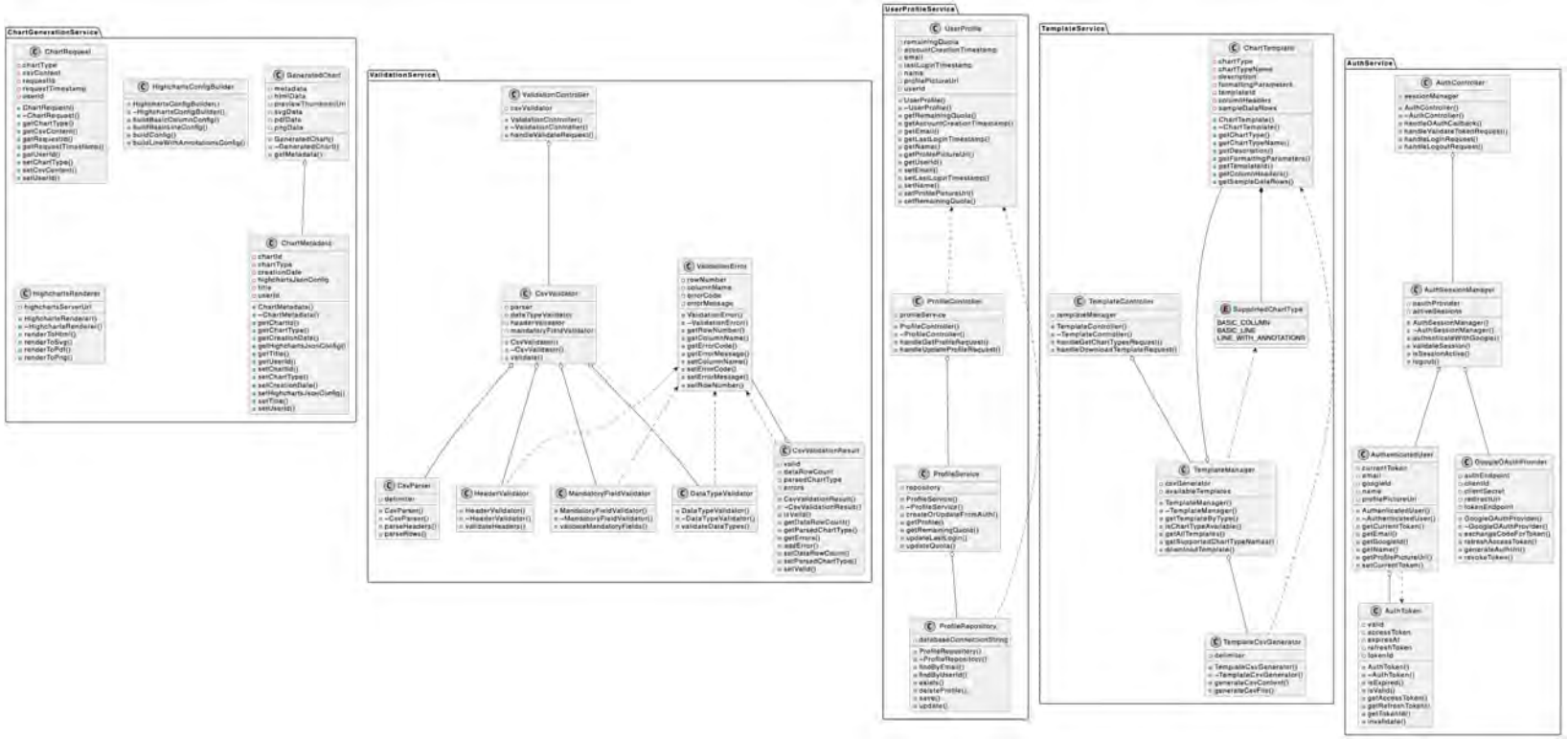
LLMs and determinism… (1/5)



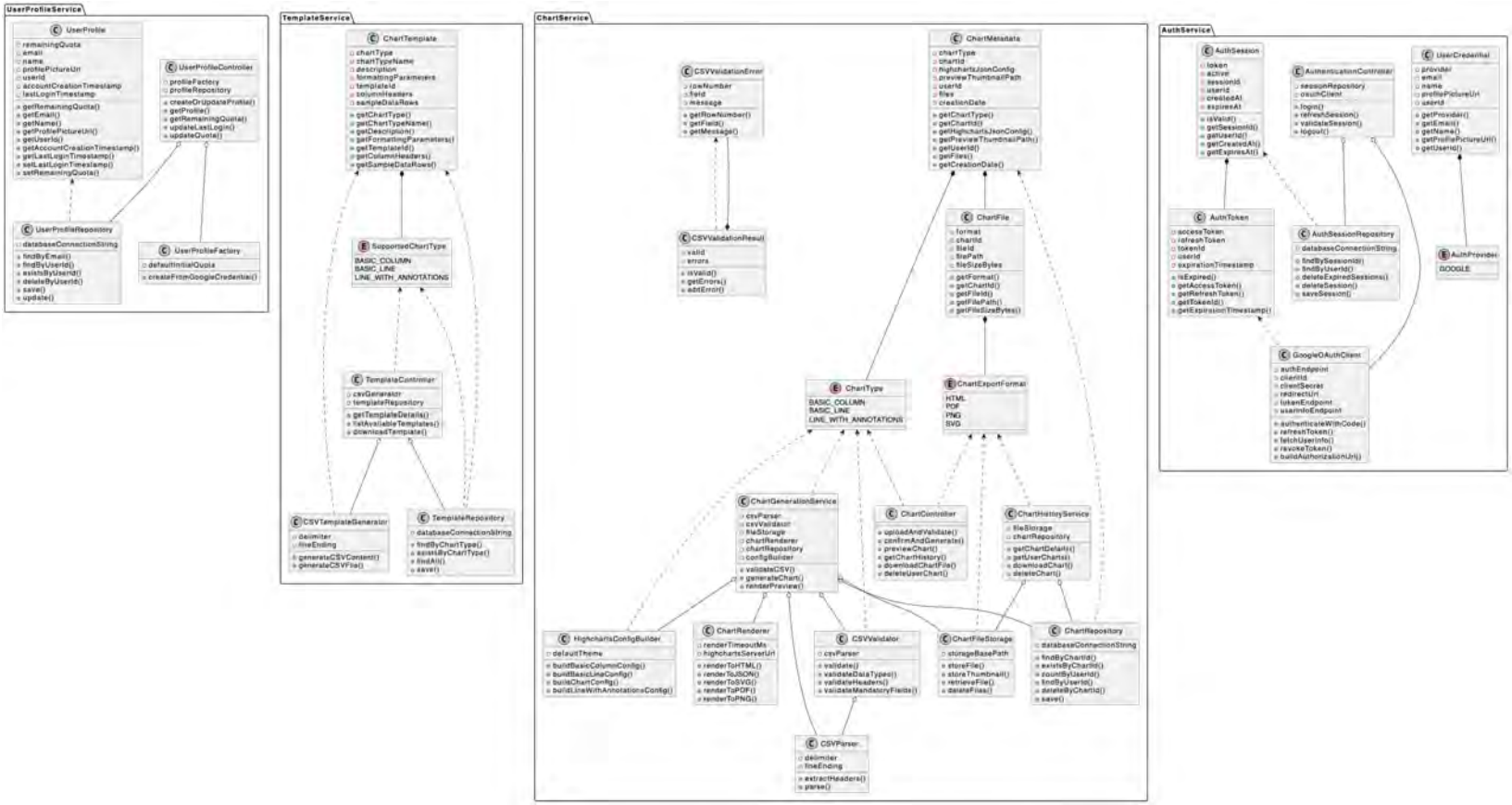
LLMs and determinism… (2/5)



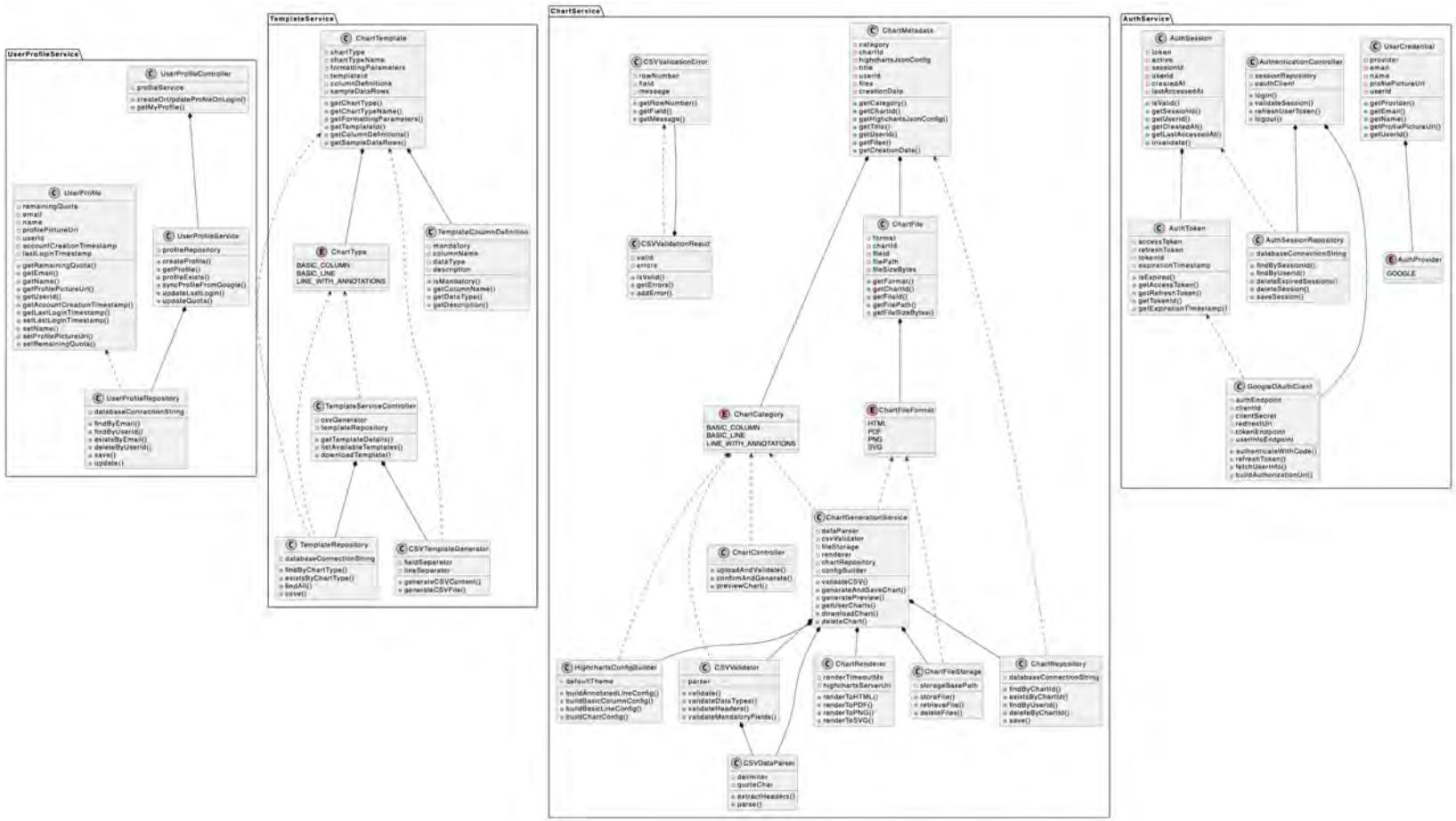
LLMs and determinism… (3/5)



LLMs and determinism… (4/5)



LLMs and determinism… (5/5)



LLMs and determinism…

AGGREGATE COMPARISON · MICROSERVICES

MYCHARTS_microservices_claude_srs_temp0

Best Worst Experiment

EXPERIMENT	# NAMESPACES	AVG CA	AVG CE	AVG I	AVG D	AVG COHESION
MYCHARTS_microservices_claude_srs_temp0_1225 ★ BEST	6	0.667	0.667	0.583	0.417	0.587
MYCHARTS_microservices_claude_srs_temp0_1226	6	0.000	0.000	0.000	1.000	0.523
MYCHARTS_microservices_claude_srs_temp0_1227 ↓ WORST	5	0.000	0.000	0.000	1.000	0.497
MYCHARTS_microservices_claude_srs_temp0_1228	4	0.000	0.000	0.000	1.000	0.588
MYCHARTS_microservices_claude_srs_temp0_1229	4	0.000	0.000	0.000	1.000	0.520
Average	5.000	0.133	0.133	0.117	0.883	0.543
Std Dev (variability)	*0.894*	*0.267*	*0.267*	*0.233*	*0.233*	*0.037*

LLMs and determinism…

AGGREGATE COMPARISON · MICROSERVICES

MYCHARTS_microservices_claude_srs_temp0.5

Best Worst Experiment

EXPERIMENT	# NAMESPACES	AVG CA	AVG CE	AVG I	AVG D	AVG COHESION
MYCHARTS_microservices_claude_srs_temp0.5_1235	5	0.000	0.000	0.000	1.000	0.457
MYCHARTS_microservices_claude_srs_temp0.5_1236	4	0.000	0.000	0.000	1.000	0.514
MYCHARTS_microservices_claude_srs_temp0.5_1237 ★ BEST	5	0.600	0.600	0.600	0.400	0.501
MYCHARTS_microservices_claude_srs_temp0.5_1238 ↓ WORST	4	0.000	0.000	0.000	1.000	0.291
MYCHARTS_microservices_claude_srs_temp0.5_1239	5	0.000	0.000	0.000	1.000	0.560
Average	4.600	0.120	0.120	0.120	0.880	0.465
Std Dev (variability)	*0.490*	*0.240*	*0.240*	*0.240*	*0.240*	*0.093*

LLMs and determinism…

AGGREGATE COMPARISON · MICROSERVICES

MYCHARTS_microservices_gpt4_srs_temp0

Best Worst Experiment

EXPERIMENT	# NAMESPACES	AVG CA	AVG CE	AVG I	AVG D	AVG COHESION
MYCHARTS_microservices_gpt4_srs_temp0_998	8	0.250	0.250	0.125	0.875	0.551
MYCHARTS_microservices_gpt4_srs_temp0_999	6	0.500	0.500	0.167	0.833	0.499
MYCHARTS_microservices_gpt4_srs_temp0_1000 ↓ WORST	5	0.000	0.000	0.000	0.779	0.232
MYCHARTS_microservices_gpt4_srs_temp0_1001	5	0.000	0.000	0.000	1.000	0.474
MYCHARTS_microservices_gpt4_srs_temp0_1208 ★ BEST	6	0.500	0.500	0.333	0.667	0.574
Average	6.000	0.250	0.250	0.125	0.831	0.466
Std Dev (variability)	*1.095*	*0.224*	*0.224*	*0.124*	*0.110*	*0.122*

AGGREGATE COMPARISON · MICROSERVICES

MYCHARTS_microservices_gpt4_frnfr_temp0.5

Best Worst Experiment

EXPERIMENT	# NAMESPACES	AVG CA	AVG CE	AVG I	AVG D	AVG COHESION
MYCHARTS_microservices_gpt4_frnfr_temp0.5_993 ★ BEST	9	1.111	1.111	0.778	0.264	0.397
MYCHARTS_microservices_gpt4_frnfr_temp0.5_994 ↓ WORST	9	0.889	0.889	0.111	0.889	0.416
MYCHARTS_microservices_gpt4_frnfr_temp0.5_995	8	0.500	0.500	0.313	0.688	0.470
MYCHARTS_microservices_gpt4_frnfr_temp0.5_996	7	0.000	0.000	0.000	0.659	0.398
MYCHARTS_microservices_gpt4_frnfr_temp0.5_997	10	1.800	1.800	0.400	0.600	0.456
Average	8.600	0.860	0.860	0.320	0.620	0.427
Std Dev (variability)	*1.020*	*0.603*	*0.603*	*0.269*	*0.203*	*0.030*

Resources

- https://davinci.softlab.ntua.gr/saasllmarch/saas_llm_arch_materials.zip
- https://davinci.softlab.ntua.gr/saasllmarch/